



VESA

VISION • EMPOWER • SKILL • ACTION

Build Projects. Build Futures.

SKILL DEVELOPMENT PROGRAM

Project Problem Statement

Project 3: Business Workflow & Operations Management Platform

Domain: Full Stack Web Development

Client

A Mid-Sized Organization Undergoing Digital Transformation

Project Duration

13 Days

from the date of assignment

*This document is issued to registered student participants of the VESA Skill Development Program,
for the purpose of hands-on, scenario-based software engineering learning.*

Project 3: Business Workflow & Operations Management Platform

Domain: Full Stack Web Development

1. Technology Stack & Concepts Covered

Students are expected to apply full-stack development and software engineering concepts while developing the solution.

The implementation may use technologies appropriate to the student's technical background, but the solution should demonstrate understanding of:

- Frontend development using React.js or an equivalent modern framework
- Component-based UI development
- Client-side routing and state management
- Backend development using Node.js and Express.js or an equivalent backend framework
- RESTful API development
- Controllers, services, middleware and business logic
- Relational or document database design
- Authentication and authorization
- Role-Based Access Control (RBAC)
- Request validation and error handling
- File/document upload and management
- Workflow and status management
- Audit logging
- Search, filtering and data aggregation
- Git and GitHub
- Frontend-backend integration
- Application deployment
- Technical documentation

Students are expected to make appropriate engineering decisions regarding architecture, database structure, API design and implementation.

2. Client

A Mid-Sized Organization Undergoing Digital Transformation

The client is a growing organization with approximately 500 employees across multiple departments.

As the organization has expanded, several internal business processes have continued to depend on email, spreadsheets, forms and messaging applications.

The management team wants to replace these fragmented processes with a centralized internal operations platform.



You have been hired as a software engineering team to design and develop the first version of this platform.

3. Background

Employees regularly need to submit requests to different departments.

For example:

- An employee needs access to a software application.
- An employee needs reimbursement for a business expense.
- An employee needs a document approved.
- An employee needs company equipment.

At present, these requests are handled through different channels.

An employee may send an email to a manager, attach a document, wait for approval, forward the approval to another department, and then communicate with the final department through another channel.

As the organization grows, this process is becoming increasingly difficult to manage.

Employees often do not know:

- Whether their request was received
- Who is currently responsible for it
- Whether approval is pending
- Why a request has been delayed
- What action is required from them
- When the request is expected to be completed

Managers and department staff face different problems.

They struggle to:

- Identify requests requiring their action
- Track pending work
- Prioritize urgent requests
- Monitor overdue requests
- Maintain a reliable history of decisions
- Understand departmental workload

Management also lacks a centralized view of operational performance.

They cannot easily determine:

- How many requests are currently open
- Which departments have the highest workload
- Which processes take the longest
- How many requests are overdue
- Where requests are getting stuck

- How efficiently requests are being resolved

The organization therefore requires a centralized web-based platform that can manage the complete lifecycle of internal business requests.

The platform must not be designed as four independent applications.

Instead, it should provide **one common request and workflow system** capable of supporting multiple business processes.

4. Business Problem

The organization has identified the following challenges:

- Internal requests are distributed across email, spreadsheets and messaging applications.
- Employees have limited visibility into request progress.
- Managers have difficulty identifying requests requiring their approval.
- Departments lack a centralized work queue.
- Approval processes are inconsistent and difficult to track.
- Important documents and supporting information are scattered across different communication channels.
- There is no reliable audit history of important decisions and actions.
- Overdue requests are difficult to identify.
- Management lacks operational analytics.
- Different business processes require different approval chains.
- Unauthorized users must not be able to perform actions outside their responsibilities.

5. Objective

Design and develop a centralized Business Workflow & Operations Management Platform that allows employees to submit internal requests and enables the appropriate managers and departments to review, approve, process and complete those requests through a controlled digital workflow.

The platform should provide a common system for:

Request Creation → Review → Assignment → Approval → Processing → Completion

The system should support different workflows for different business processes while maintaining a consistent user experience.

The solution should improve:

- Operational visibility
- Request tracking
- Approval management
- Department coordination
- Accountability
- Process consistency
- Security

- Decision traceability

The platform should also provide administrators and management with meaningful operational insights through dashboards and reports.

6. Users & Roles

The client has identified the following primary users.

Role	Key Responsibilities
Employee	Create requests, provide required information, upload supporting documents, track requests, respond to clarification requests
Reporting Manager	Review requests submitted by team members, approve/reject requests, request clarification
Department Staff	Process requests assigned to their department, update progress, add notes, complete operational tasks
Department Head / Director	Review requests requiring higher-level approval and make approval decisions
System Administrator	Manage users, departments, roles, request types, approvers, workflows and system configuration
Operations Manager	Monitor organizational workload, SLA performance, bottlenecks and operational metrics

7. Core Business Processes

The following four processes are mandatory.

Students must implement all four within the common workflow platform.

7.1 Software Access Request

An employee requires access to an organizational software application.

Request Information

The request should capture, at minimum:

- Request ID
- Employee
- Department
- Application / Software Name
- Requested Access Level
- Business Justification
- Required Date
- Priority
- Additional Comments
- Supporting Document, if required

Example

Employee: Aarav Sharma

Department: Product Engineering

Application: Project Management System

Access Level: Standard User

Business Justification: Required for project coordination

Required Date: 20 August

Approval Flow

Employee → Reporting Manager → IT Administrator → Completed

The Reporting Manager validates the business requirement.

The IT Administrator verifies the technical/access requirement and processes the request.

Rules

- An employee cannot approve their own request.
- The Reporting Manager can approve, reject or request clarification.
- IT can process the request only after managerial approval.
- A rejected request must record the reason.
- The request must maintain its complete history.

7.2 Expense Reimbursement

An employee needs reimbursement for an eligible business expense.

Request Information

The request should capture:

- Request ID
- Employee
- Department
- Expense Category
- Expense Date
- Amount
- Description
- Business Purpose
- Receipt / Supporting Document
- Priority
- Submission Date



Example

Employee: Priya Mehta

Department: Marketing

Category: Client Meeting

Amount: ₹4,850

Business Purpose: Client meeting and travel

Receipt: Attached

Approval Flow

Employee → Reporting Manager → Finance Officer → Reimbursement Processing → Completed

Rules

- The employee cannot approve their own reimbursement.
- The manager reviews the business justification.
- Finance verifies the expense and supporting documents.
- Finance may request additional information.
- Rejected expenses must contain a rejection reason.
- The final reimbursement status must be recorded.

7.3 Document Approval

An employee needs formal approval for an organizational document.

This could include:

- Internal policy
- Process document
- Client-facing document
- Technical document
- Department proposal

Request Information

The request should capture:

- Request ID
- Document Title
- Document Type
- Department
- Document Version
- Description
- Submitted By
- Submission Date
- Document Attachment



- Priority
- Approval Deadline

Example

Document: Customer Data Handling Procedure

Type: Internal Policy

Department: Operations

Version: 1.0

Submitted By: Neha Verma

Approval Flow

Employee → Department Manager → Department Director → Final Approval → Completed

Rules

- Each approver must make an explicit decision.
- An approver may approve, reject or request changes.
- A rejection must include a reason.
- A request for changes should return the document to the appropriate previous stage.
- The system must retain the document version and approval history.
- The system must record who approved each stage and when.

7.4 Equipment Request

An employee requires organizational equipment.

Examples include:

- Laptop
- Monitor
- Keyboard
- Mouse
- Headset
- Other approved equipment

Request Information

The request should capture:

- Request ID
- Employee
- Department
- Equipment Type
- Quantity
- Business Justification
- Required Date



- Priority
- Additional Information

Example

Employee: Rohan Patil

Department: Software Engineering

Equipment: External Monitor

Quantity: 1

Justification: Required for development and testing

Required Date: 25 August

Approval Flow

Employee → Reporting Manager → IT / Administration → Procurement or Inventory → Completed

The organization may use different processing paths depending on equipment availability.

Rules

- Manager approval is required before operational processing.
- The processing department must indicate whether the item is available.
- If unavailable, the request may move to procurement.
- The system should record the final allocation or procurement status.
- The request must not be marked completed without an appropriate completion action.

8. Common Request Lifecycle

Although each business process has its own workflow, all requests should follow a controlled lifecycle.

A typical lifecycle is:

Submitted → Under Review → Approval Pending → Approved / Rejected → Processing → Completed

Students may introduce additional states where required.

For example: Changes Requested or Cancelled.

However, every status must have a clearly defined purpose.

Students must determine:

- Who can change a status
- Under what conditions it can change
- Which status can come next
- What happens when a request is rejected
- What happens when additional information is required

9. Request Management

Every request should have a unique identifier.

Example: **REQ-2026-00124**

The request details page should provide a complete view of the request, including:

- Request information
- Current status
- Current owner/assignee
- Priority
- Submitted date
- Required date
- SLA status
- Comments
- Attachments
- Approval decisions
- Activity history

The user should be able to understand the current state of a request without searching through emails or messages.

10. Approval & Action Management

The platform must provide role-specific actions.

Depending on the request and current workflow stage, a user may be allowed to:

- Approve
- Reject
- Request Changes
- Assign
- Reassign
- Start Processing
- Update Progress
- Complete
- Add Comment

The backend must enforce these permissions.

Hiding a button on the frontend is not considered authorization.

For example, even if an employee cannot see an “Approve” button, the backend must still reject an unauthorized approval request sent directly to the API.

11. Comments & Clarification

Users should be able to communicate within a request.

For example:

Finance Officer: Please upload the original receipt.

The employee can then provide the required document.

The system should maintain this communication as part of the request history.

Students should consider:

- Who can comment?
- Who can view comments?
- When can comments be added?
- Can comments be edited?
- Should important actions require comments?

These decisions should be documented as assumptions.

12. Document & Attachment Management

Certain requests require supporting documents.

Examples:

- Expense receipt
- Approval document
- Technical specification
- Equipment quotation

The system should allow authorized users to upload and access relevant attachments.

Students should consider:

- File type validation
- File size limits
- Access control
- File naming
- Secure storage
- Unauthorized document access

Sensitive documents must not be publicly accessible without appropriate authorization.

13. SLA & Overdue Requests

The organization wants to monitor how quickly requests are processed.

Each request type should have a target processing period.

Request Type	Target
Software Access	24 hours
Expense Reimbursement	48 hours
Document Approval	72 hours
Equipment Request	72 hours

The system should identify requests that are:

- Within SLA
- Approaching SLA
- Overdue
- Completed within SLA
- Completed after SLA

Students should calculate SLA status using actual request timestamps rather than storing a manually entered label.

14. Dashboard & Operational Monitoring

The platform should provide dashboards for appropriate users.

Employee Dashboard

Should provide information such as:

- My active requests
- Requests awaiting action
- Recently completed requests
- Overdue requests
- Request status summary

Manager Dashboard

Should provide:

- Pending approvals
- Team requests
- Requests requiring attention
- Recently approved/rejected requests

Department Dashboard

Should provide:

-
- Assigned requests
 - Pending work
 - In-progress requests
 - Overdue requests
 - Workload by request type

Operations / Admin Dashboard

Should provide meaningful organization-level metrics such as:

- Total requests
- Open requests
- Pending approvals
- In-progress requests
- Completed requests
- Overdue requests
- Requests by department
- Requests by category
- Average processing time
- SLA performance

Students should avoid displaying meaningless statistics simply to fill dashboard space.

15. Search & Filtering

The platform should provide efficient ways to find requests.

Users should be able to search or filter based on appropriate permissions using attributes such as:

- Request ID
- Request type
- Employee
- Department
- Status
- Priority
- Assigned person
- Date range
- SLA status

The backend should support appropriate filtering rather than relying entirely on frontend filtering.

16. Notifications

The system should notify users when an important action requires their attention.

Examples:

- Request submitted
- Approval required
- Request approved
- Request rejected
- Changes requested
- Request assigned
- Request completed
- SLA approaching
- Request overdue

For this project, **in-app notifications are sufficient**.

Email notifications may be implemented as an optional enhancement.

17. Audit Trail

The system must maintain an audit history for important actions.

Example:

REQ-2026-00124

```
10:15 AM – Request submitted by Aarav Sharma
10:20 AM – Assigned to Reporting Manager
11:05 AM – Manager approved request
11:07 AM – Assigned to IT Administrator
01:42 PM – IT Administrator started processing
03:15 PM – Access granted
03:16 PM – Request marked completed
```

The audit trail should answer:

- Who performed the action?
- What action was performed?
- Which request was affected?
- When did it happen?
- What was the relevant decision or status?

Students should design an appropriate audit-log structure.

18. Authentication & Authorization

The application must implement secure authentication.

At minimum:

- User registration or administrator-created accounts
- Login
- Secure password handling
- Protected routes
- Session/JWT-based authentication
- Logout

The system must implement role-based authorization.

For example, an Employee should not be able to:

- Access another employee's private requests
- Approve requests
- Modify system configuration
- Access administrative analytics

A Manager should not automatically receive administrative permissions.

A user should only be able to perform actions appropriate to their role **and the current workflow stage**.

19. Business Rules & Constraints

The following constraints are mandatory considerations:

- An employee cannot approve their own request.
- A request cannot skip mandatory approval stages.
- Only authorized users can perform workflow actions.
- Rejected requests must contain a reason.
- Requests requiring changes must return to an appropriate stage.
- A request cannot be marked completed without completing its required processing stage.
- Users must not access requests outside their authorization scope.
- Supporting documents must only be accessible to authorized users.
- SLA status must be calculated from relevant timestamps.
- Important workflow actions must be recorded in the audit trail.
- The system must prevent invalid status transitions.
- Deleting important business records should not destroy their historical accountability.
- Backend authorization must be enforced independently of frontend UI restrictions.

20. Data & Relationship Design

The client expects the system to maintain relationships between organizational entities.

At minimum, students should consider data for:

- Users
- Roles
- Departments
- Request Types
- Requests
- Workflow Stages
- Approvals
- Assignments
- Comments
- Attachments
- Notifications
- SLA configuration
- Audit Logs

The database schema is intentionally not provided.

Students must analyze the requirements and design an appropriate data model.

The quality of the database design will be evaluated based on:

- Relationships
- Data consistency
- Appropriate constraints
- Avoidance of unnecessary duplication
- Maintainability
- Ability to support the required workflows

21. API & Backend Expectations

The backend should expose well-structured APIs for the application.

Students should consider APIs for:

- Authentication
- Users
- Departments
- Requests
- Request types
- Approvals
- Assignments

- Comments
- Attachments
- Notifications
- Dashboard/analytics
- Audit history

Students should organize backend code appropriately using concepts such as:

Routes → Controllers → Services → Data Layer

where appropriate for their chosen architecture.

Business rules should not be implemented exclusively in the frontend.

22. Non-Functional Requirements

The client expects the solution to demonstrate professional engineering practices.

Quality	Expectation
Security	Sensitive information and documents must be protected.
Reliability	Invalid requests should be handled gracefully without crashing the application.
Maintainability	The application should have a logical structure and reusable components.
Usability	Users should be able to understand what action is required from them.
Scalability	The design should reasonably support growth in users, requests and departments.
Performance	Common operations such as listing, searching and filtering requests should remain responsive.
Auditability	Important business actions must be traceable.

23. Open-Ended Engineering Questions

Before implementation, students should think about:

- How will you represent different workflows in your database?
- How will the system determine who is allowed to approve a request?
- What happens if an approver rejects a request?
- What happens if an approver requests changes?
- How will you prevent an employee from approving their own request?
- How will SLA deadlines be calculated?
- What happens when a request becomes overdue?
- How will the system handle a user changing departments?
- How will you ensure users cannot access another user's confidential documents?
- What happens if two users attempt to update the same request?
- How would your architecture change if the organization grew from 500 to 50,000 employees?

There may be multiple valid engineering solutions.

Students should be able to justify the decisions they make.

24. Creative Freedom

The core requirements above are mandatory.

Students may introduce additional features that improve the system, provided those features do not compromise the core requirements.

Possible extensions include:

- Email notifications
- Configurable workflow builder
- Automatic escalation for overdue requests
- Advanced analytics
- Exportable reports
- Bulk request processing
- Request templates
- Department-specific dashboards
- Search optimization
- Activity notifications
- Approval delegation
- Workflow versioning
- Advanced role/permission management

These are suggestions, not requirements.

Original, practical engineering improvements will be rewarded.

Students should explain why their additional feature is useful rather than adding features simply for complexity.

25. Expected Engineering Deliverables

The final submission should demonstrate the complete software engineering lifecycle.

Students are expected to produce:

1. Requirements

- Problem understanding
- Stakeholders
- Functional requirements
- Non-functional requirements
- Business rules
- Assumptions

2. System Design

- Architecture diagram
- User/role design
- Workflow design
- Database/ER diagram
- API design

3. Working Application

- Frontend
- Backend
- Database
- Authentication
- Authorization
- Four mandatory business workflows
- Dashboard
- Audit trail
- SLA tracking

4. Testing

Students should test:

- Authentication
- Authorization
- Request creation
- Approval flows
- Invalid transitions
- Validation
- Access restrictions
- Edge cases

5. Deployment (Optional)

The application should be deployed so that the reviewer can access the working system.

26. Submission

The mandatory VESA submission will require:

- GitHub Repository Link
- Deployed Application Link
- README.md
- Database / ER Diagram
- Architecture Diagram
- API Documentation
- Complete VESA Google Form submission

The GitHub repository should contain a clean and understandable project structure.

27. README Requirements

The README must contain:

- Project name
- Problem understanding
- Proposed solution
- Stakeholders and roles
- Major features
- Technology stack
- System architecture
- Database design
- API overview
- Workflow explanation
- Business rules
- Setup instructions
- Environment variables/configuration
- Assumptions
- Testing approach
- Deployment information
- Future improvements

Do not copy the VESA problem statement into your README.

Explain the problem and your engineering solution in your own words.

28. Evaluation Rubric — 100 Marks

Evaluation Criteria	Marks
Problem Understanding & Stakeholder Analysis	10
Requirement Engineering & Business Rules	10
System Architecture & Database Design	10
Backend APIs & Business Logic	15
Frontend Development & User Experience	10
Authentication, Authorization & Security	10
Workflow, Approval & SLA Implementation	10
Testing, Validation & Error Handling	5
Documentation & Technical Communication	10
GitHub Quality, Deployment & Professionalism	5
Creativity & Engineering Decisions	5
Total	100

29. What We Expect From You

You are not expected to receive a list of pages and simply convert them into code.

You are being placed in the role of a **software engineering team working for a real client**.

Your responsibility is to:

- **Understand the business problem.**
- **Identify the requirements.**
- **Design the solution.**
- **Model the data.**
- **Define the workflows.**
- **Build the application.**
- **Secure the system.**
- **Test your decisions.**
- **Document your work.**
- **Deploy the solution.**

Most importantly, you should be able to explain why you built the system the way you did.

A working application is important.

But for this final project, your engineering thinking matters just as much as your code.



30. Final Project Mindset

Project 3 is designed to simulate the type of problem a software services team may receive from an organizational client.

The client does not tell you:

- Which database tables to create
- Which APIs to write
- Which React components to build
- How to structure your folders
- Which libraries to use
- Exactly how your UI should look

Instead, the client gives you a business problem, stakeholders, processes, rules and constraints.

Your job is to convert that information into a reliable software solution.

Don't build what you think looks impressive. Build what solves the client's problem.

This is the final project of the VESA Skill Development Program.

Build the system like an engineer. Document it like a professional. Defend your decisions like a software developer.

VESA

Vision • Empower • Skill • Action
Build Projects. Build Futures.